

Requirement-Grounded LLM Agents for Functional Testing of React Applications

A Controlled Study Protocol, Benchmark Specification, and Artifact Audit

Abdurrahman Khan

Barkatullah University Institute of Technology, Bhopal, India
Independent Research

September 2026

Abstract

Large language models (LLMs) can generate executable browser tests, but executability does not show that a test represents intended behavior. This paper defines a controlled experiment for measuring whether structured natural-language requirements improve LLM-generated functional tests for React applications. The benchmark design comprises three applications, 24 requirements, 50 atomic behaviors, and 12 single-fault mutant specifications. Baseline and requirement-grounded agents are matched on model, browser access, generation budget, repair budget, and execution environment; ten independent replicates per condition and application yield 60 planned generation sessions. Frozen suites are to be replayed on clean and mutant builds and assessed for requirement coverage, behavior coverage, generation and execution success, defect detection, false positives, invalid tests, and traceability. An audit of the supplied artifact found only an earlier protocol PDF and a minimal LaTeX stub: the applications, agent, mutant patches, reference tests, run records, and logs were absent, while Node.js, Playwright, and Chromium were unavailable. Consequently, no empirical result or claim of treatment superiority is reported. The present contribution is a versioned experimental specification, a 50-behavior coding inventory, a 12-mutant pre-experiment specification, machine-readable record schemas, and deterministic validation and analysis scripts. The protocol makes the intended study auditable, but empirical conclusions require execution after the missing artifact is restored.

1 Introduction

Functional testing of modern web applications is difficult because user-visible behavior emerges from navigation, authentication, asynchronous rendering, persistent state, validation rules, and role-dependent controls. End-to-end tests must establish realistic preconditions, exercise workflows through the rendered interface, and decide whether the observed outcome is correct. The last step is the test-oracle problem: automation can perform actions reliably while still asserting the wrong behavior [4].

LLMs offer a plausible way to reduce the manual effort of writing functional tests. They can translate descriptions into code, use browser tools, interpret failures, and revise malformed tests. TestPilot, for example, showed that execution feedback can support LLM-based generation and repair of JavaScript unit tests [1]. Browser-level testing is a harder setting. A generated suite may overfit to visible controls, omit negative cases, hallucinate selectors, assume unavailable state, or infer its oracle from the current implementation. Such a suite can execute successfully without testing what the system is supposed to do.

Requirements may supply the missing semantic anchor. They identify actors, preconditions, prohibited outcomes, state transitions, and observable acceptance conditions. Earlier work has investigated deriving tests from natural-language requirements, but tool availability and methodological reporting remain uneven [2, 3]. The empirical question is therefore not whether LLMs can emit test code, but whether explicit requirement access improves the relevance and fault-detection capability of an otherwise matched browser-testing agent.

This paper preserves that question in a controlled design. It compares a baseline agent that explores an application without the approved requirement inventory with a grounded agent that receives the versioned requirements and must retain requirement identifiers. Both conditions use the same application, model, browser tools, accounts, budgets, and repair policy. The planned benchmark contains TaskBoard, InventoryDesk, and HelpQueue; 24 requirements; 50

atomic behaviors; and 12 single-fault mutants. Ten replicates for each application–condition pair produce 60 planned generation sessions.

The supplied workspace did not contain the implementation or empirical records needed to execute this design. This revision therefore reports a study protocol and artifact audit rather than invented findings. Its concrete contributions are:

1. a controlled requirement-grounding treatment for browser-based LLM test generation;
2. a versioned benchmark specification containing 24 requirements and a predeclared 50-behavior inventory;
3. a transparent specification of 12 single-fault mutants and the evidence required to count a detection;
4. machine-readable run-record and analysis conventions connecting requirement, behavior, test, execution, and result; and
5. a reproducibility audit that distinguishes available inputs, newly formalized protocol assets, and missing empirical evidence.

2 Background and Motivation

2.1 LLM-generated tests

LLMs have been evaluated for unit-test generation, repair, and related software-engineering tasks. TestPilot generated JavaScript unit tests from function implementations and documentation and used failure feedback for repair [1]. That evaluation also illustrates the need to record model identity, prompt context, repeated runs, and executable outcomes. A browser test adds interaction sequences, state reset, authentication, selectors, and UI-level oracles, making code generation only one stage of the process.

2.2 Requirements and test oracles

Requirements-based testing derives test conditions from intended behavior and maintains links between specifications and verification artifacts. Garousi et al. mapped 67 studies on NLP-assisted testing and reported limited downloadable tool support and incomplete descriptions of NLP components [2]. Mustafa et al. likewise reviewed automated test generation from requirements [3]. LLMs can support a richer transformation, but ambiguity remains: requirements may omit preconditions, combine behaviors, or leave an outcome implicit. The agent must record ambiguity rather than silently inventing acceptance criteria.

2.3 Browser agents and execution evidence

Tool-using agents interleave observations and actions, as exemplified by ReAct [5]. WebArena shows that long-horizon tasks in stateful web environments remain challenging even for capable agents [6]. Playwright supplies semantic locators, auto-waiting, assertions, and execution traces. Its documentation recommends user-facing locators such as roles and labels over DOM-coupled selector chains [8]; traces retain actions, screenshots, DOM snapshots, console output, and network evidence useful for diagnosis [9]. These facilities improve observability but do not by themselves validate the oracle.

2.4 Research gap

Prior work establishes that LLMs can generate tests, that natural-language artifacts can support test derivation, and that language-guided agents can operate websites. It does not establish the causal effect of supplying approved requirements to a matched LLM agent performing functional testing of controlled React applications. The present design isolates that treatment and measures both benefits and costs, including invalid tests and false positives.

3 Research Questions

RQ1

How does explicit requirement grounding change the number, diversity, validity, and oracle content of functional tests generated by an LLM agent?

RQ2

How does grounding change coverage of approved requirements and predeclared atomic behaviors?

RQ3

How does grounding change detection of documented single-fault mutants?

RQ4

How does grounding affect generation success, execution success, invalid tests, false positives, and requirement-to-test traceability?

RQ5

Which requirement-interpretation, workflow, selector, state, authorization, oracle, execution, and reporting failures occur in each condition?

Two-sided hypotheses are used for the primary outcomes because grounding may help, have no effect, or harm performance. The null hypotheses state that the median paired differences in requirement coverage and defect detection are zero.

4 Experimental Methodology

4.1 Design and unit of analysis

The experiment is a matched repeated-run comparison. Each application is evaluated under baseline and grounded conditions with ten independently generated suites per condition. The application-run suite is the analysis unit; tests within a suite are nested observations and are not treated as independent replicates. Application identity is a blocking factor, and replicate labels are assigned before randomized interleaving.

Let A_0 be a clean application, R its approved requirements, B its atomic behaviors, and A_j a single-fault mutant. The agent $G(c, A_0)$ creates a suite T under condition $c \in \{0, 1\}$, where $c = 0$ is baseline and $c = 1$ is requirement-grounded. After bounded clean-build repair, T is frozen and replayed unchanged against A_0 and every A_j for that application.

4.2 Controlled variables

The model and snapshot, inference settings, application commit, browser and viewport, accounts, data seed, timeout, maximum turns, output-token ceiling, repair attempts, locator guidance, and evidence collection must be identical across conditions. The grounded prompt adds only the approved requirements and traceability instructions. The baseline receives the URL, neutral application label, accounts, reset command, and tool documentation, but not requirement text, behavior definitions, mutant metadata, or source code.

4.3 Randomization and blinding

Run order is generated from the recorded seed 20260929 and interleaved within application. Condition labels are replaced with opaque identifiers before manual semantic coding. Prompts, model responses, tool calls, generated files, suite hashes, traces, screenshots, logs, environment manifests, and coding decisions are retained.

5 System and Agent Architecture

Figure 1 shows the shared pipeline. The grounded branch adds requirement normalization and agent-produced trace links; execution and classification remain common.

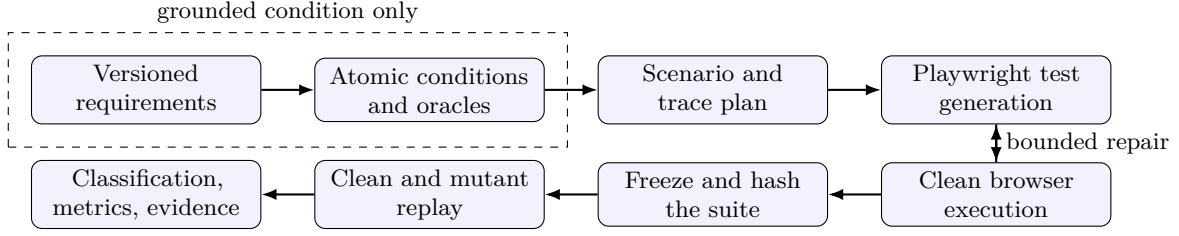


Figure 1: Requirement-grounded generation and common execution pipeline. The baseline enters at scenario generation without the two dashed-box inputs.

The requirement processor retains the original statement and emits actor, preconditions, action, input partitions, expected outcome, prohibited outcome, state transition, and ambiguity notes. A generated test is acceptable only if it includes setup, at least one meaningful browser action, and at least one observable assertion. Repairs may correct syntax, imports, selectors, and timing, but may not delete the intended oracle, weaken a precise assertion to element existence, skip a test, or exceed the shared budget.

6 React Application Benchmark

Table 1 summarizes the three intended applications. They are controlled research applications, not a claim of representing the full complexity of production systems.

Table 1: Benchmark scope and fixed counts.

Application	Main workflows	Requirements	Behaviors	Mutants
TaskBoard	authentication; task creation, validation, editing, completion, search, filtering, confirmation, and persistent deletion	8	17	4
InventoryDesk	authentication; viewer/manager restrictions; item creation, validation, editing, filtering, deletion, and confirmations	8	16	4
HelpQueue	submission; validation; ticket access; search/filter; role operations; transitions; navigation; confirmations	8	17	4
Total		24	50	12

Each clean application must pass a researcher-written reference acceptance suite before entering the experiment. Every mutant must build, preserve unrelated reference behavior, and be killed by at least one reference test for the affected requirement. These gates cannot be verified until the missing application sources and patches are restored.

7 Requirements and Behavioral Model

The complete statements are stored in `requirements/requirements.json`. Tables 2–4 give a compact inventory. Identifiers are immutable within a dataset version.

Table 2: Approved TaskBoard requirements.

ID	Observable intent
TB-AUTH-01	Valid member credentials reach the task board and expose the member display name.
TB-AUTH-02	Invalid credentials remain on sign in and show a non-disclosing error.
TB-TASK-01	Create a non-empty task and retain it after reload.
TB-TASK-02	Reject empty and whitespace-only titles without adding a task.
TB-TASK-03	Edit a task title and retain the edit after reload.
TB-TASK-04	Complete a task and filter active versus completed tasks.
TB-TASK-05	Search titles case-insensitively and show a no-match state.
TB-TASK-06	Require deletion confirmation; cancel retains and confirm removes persistently.

Table 3: Approved InventoryDesk requirements.

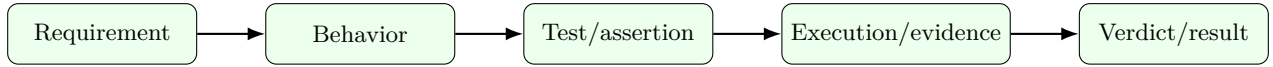
ID	Observable intent
ID-AUTH-01	Valid manager authentication reaches inventory; protected unauthenticated access redirects.
ID-ROLE-01	Viewers can inspect/filter but receive no item-management controls.
ID-ITEM-01	Managers create items with unique SKU, name, and integer quantity ≥ 0 .
ID-ITEM-02	Reject duplicate SKU, empty name, negative quantity, and non-integer quantity with field feedback.
ID-ITEM-03	Edit item name and quantity and retain changes after reload.
ID-ITEM-04	Confirm deletion and remove only the selected item.
ID-FILT-01	Low-stock results contain exactly quantities below the displayed threshold.
ID-NOTI-01	Create, update, and delete show operation-specific confirmations.

Table 4: Approved HelpQueue requirements.

ID	Observable intent
HQ-TICK-01	Submit a complete ticket and display its identifier.
HQ-TICK-02	Missing required fields prevent submission and produce field feedback.
HQ-TICK-03	Requesters see only owned tickets and can open their details.
HQ-SRCH-01	Search identifier/subject case-insensitively; category and status filters are conjunctive.
HQ-ROLE-01	Agents see all tickets and use only displayed legal status transitions.
HQ-ROLE-02	Requesters neither see nor invoke agent-only status operations.
HQ-NAV-01	A nonexistent ticket shows a not-found view with a route to the list.
HQ-NOTI-01	Creation and status-change confirmations identify the ticket or resulting status.

The 50 atomic behaviors prevent a test from receiving full credit merely because it mentions a requirement. The decomposition includes 5 authentication, 8 validation/boundary, 7 authorization, 6 persistence/state-transition, 8 search/filter, 5 navigation/confirmation, 5 create/edit/delete, and 6 oracle/notification/empty-state behaviors. A behavior is covered only when a valid test establishes its precondition, performs its action, and asserts its observable result. The complete mapping is versioned in `behaviors/behaviors.json`.

Figure 2 defines the evidence chain. A broken link prevents a requirement-level empirical claim.

**Figure 2:** Required traceability chain from intended behavior to reported result.

8 Experimental Conditions

The shared prompt defines the role, allowed tools, output schema, locator policy, evidence requirements, and prohibited actions. The baseline addendum asks the agent to explore visible workflows and record assumptions. The grounded addendum supplies the versioned requirements, requires atomic scenarios and requirement-derived assertions, preserves IDs, and records ambiguities. Both prompts are included verbatim under `prompts/`. Prompt length is reported rather than padded with irrelevant baseline text.

Figure 3 makes the treatment boundary explicit. Requirements and agent-produced traces are the only intended additions to the grounded condition. Application access, model settings, browser tools, accounts, generation budget, repair budget, and execution policy remain shared.

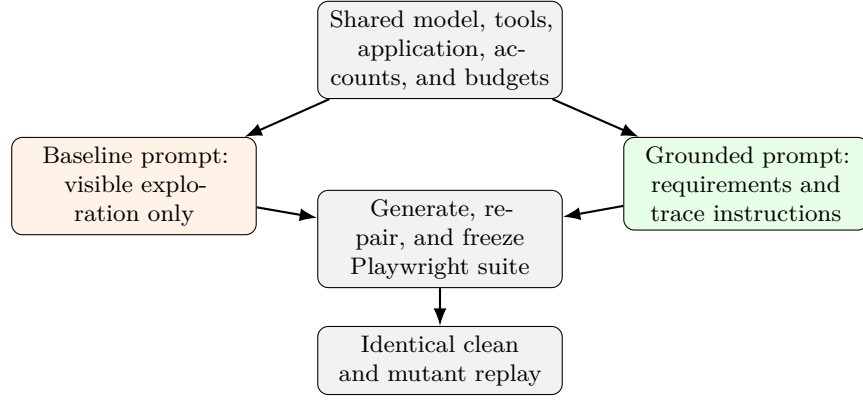


Figure 3: Matched baseline and requirement-grounded conditions. The two prompt branches differ only in approved requirement access and traceability instructions.

Table 5: Experimental configuration and audit status. Null entries must be fixed before execution.

Factor	Fixed policy	Current recorded value
Applications	TaskBoard, InventoryDesk, HelpQueue	fixed names; implementations absent
Conditions	baseline and requirement-grounded	fixed
Replicates	10 per application and condition	60 planned sessions
Model/settings	identical snapshot and inference settings	not supplied
Generation/repair budget	identical across conditions	not supplied
Browser	Chromium with fixed viewport, locale, timezone, and timeouts	browser absent
Runner	Playwright with traces, screenshots, console and network logs	runtime absent
Randomization	interleaved using recorded seed	seed 20260929
Data reset	deterministic fixture and health check before every session	implementation absent
Failure reruns	two reruns on unchanged build and fixture	fixed policy

9 Mutant Specification

Table 6 lists the pre-experiment mutant specification formalized from the protocol’s declared fault classes. The supplied workspace did not include the original mutant manifest or patches; therefore these rows are not represented as implemented mutants. Their identifiers, source patches, and reference-test gates must be reconciled with the restored artifact before data collection.

Table 6: Twelve single-fault mutant specifications. All implementation statuses are currently missing.

ID	App.	Requirement	Fault	Expected observable violation
TB-M01	TB	TB-TASK-02	Whitespace accepted	A whitespace title creates a task.
TB-M02	TB	TB-TASK-03	Edit not persisted	The title reverts after reload.
TB-M03	TB	TB-TASK-05	Case-sensitive search	A differently cased match is omitted.
TB-M04	TB	TB-TASK-06	Wrong task deleted	Confirmation removes an unrelated task.
ID-M01	ID	ID-ITEM-02	Negative quantity accepted	An invalid item is created.
ID-M02	ID	ID-ITEM-03	Quantity edit not persisted	The quantity reverts after reload.
ID-M03	ID	ID-FILT-01	Threshold off by one	Quantity equal to threshold is included.
ID-M04	ID	ID-ROLE-01	Viewer privilege leak	A viewer receives a management action.
HQ-M01	HQ	HQ-TICK-02	Empty description accepted	Incomplete submission succeeds.
HQ-M02	HQ	HQ-SRCH-01	Filters use OR	A ticket matching one filter is shown.
HQ-M03	HQ	HQ-ROLE-02	Requester changes status	A restricted transition succeeds.
HQ-M04	HQ	HQ-NAV-01	Missing not-found route	Invalid ID renders a blank detail view.

A suite detects a mutant only when an unchanged frozen test passes (or reaches its intended oracle) on the clean build, fails reproducibly on the mutant for the documented violation, and the failure is not invalid, flaky, or infrastructural. The detecting test IDs and evidence paths are mandatory run-record fields.

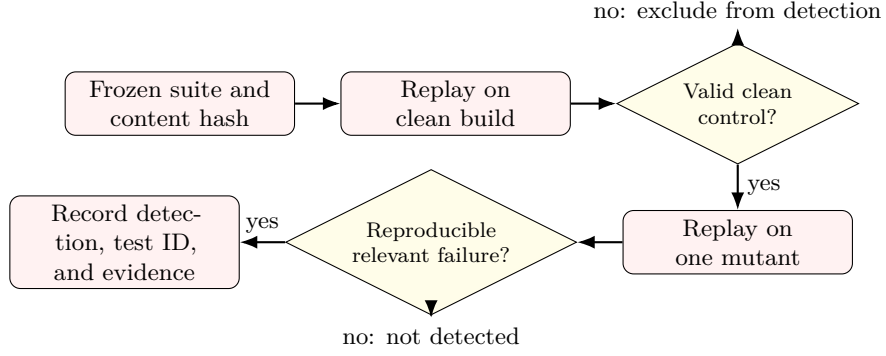


Figure 4: Mutant-detection decision workflow. A mutant is counted only with a valid clean control and a reproducible, fault-relevant failure of the unchanged suite.

10 Metrics

Let T be all generated tests in a suite, $V \subseteq T$ valid tests, $E \subseteq T$ tests that complete execution, R requirements, B behaviors, and D non-equivalent mutants. The denominators are application-specific for RC, FC, and DDR and suite-specific for test-level measures.

Requirement coverage (RC). For $I_R(r) = 1$ only when a valid test performs the relevant behavior and asserts an observable requirement outcome,

$$RC = \frac{\sum_{r \in R} I_R(r)}{|R|} \times 100\%. \quad (1)$$

Functional/behavior coverage (FC). For $I_B(b) = 1$ only when an atomic behavior’s setup, action, and oracle are present,

$$FC = \frac{\sum_{b \in B} I_B(b)}{|B|} \times 100\%. \quad (2)$$

Generation success rate (GSR).

$$GSR = \frac{\text{sessions producing at least one loadable test}}{\text{all generation sessions}} \times 100\%. \quad (3)$$

Execution success rate (ESR).

$$ESR = \frac{|E|}{|T|} \times 100\%. \quad (4)$$

A meaningful assertion failure on a mutant counts as execution success because the test produced a verdict.

Defect detection rate (DDR). For $I_D(d) = 1$ when at least one valid reproducible clean-controlled failure detects mutant d ,

$$DDR = \frac{\sum_{d \in D} I_D(d)}{|D|} \times 100\%. \quad (5)$$

False-positive rate (FPR). Following the protocol, FPR is the share of valid reproducible failure reports attributable to the clean build:

$$FPR = \frac{|F_0|}{\max(1, |F_{all}|)} \times 100\%. \quad (6)$$

Invalid, flaky, infrastructural, and confirmed clean-application defects are excluded from F_0 . The clean-suite pass proportion is also reported because this denominator can be unstable when few failures occur.

Invalid-test rate (ITR).

$$ITR = \frac{|T \setminus V|}{|T|} \times 100\%. \quad (7)$$

Requirement traceability (RT). For grounded suites, a complete trace links a valid requirement ID to a substantive assertion, verdict, and evidence artifact:

$$RT = \frac{\text{valid grounded tests with a complete correct trace}}{|V|} \times 100\%. \quad (8)$$

Baseline tests are mapped post hoc for coverage comparison, but this mapping is not reported as agent-produced traceability.

Table 7: Metric denominators and interpretation.

Metric	Unit	Denominator	Interpretation
RC	suite	requirements for the application	requirements exercised by a valid action and relevant oracle
FC	suite	atomic behaviors for the application	behaviors with qualifying setup, action, and assertion
GSR	condition/application	attempted generation sessions	sessions producing at least one loadable test
ESR	suite	all generated tests	tests completing execution and producing a verdict
DDR	suite	non-equivalent mutants for the application	mutants detected under the clean-control rule
FPR	suite/condition	all valid reproducible failure reports	share attributable to the clean build
ITR	suite	all generated tests	tests unable to provide a meaningful verdict
RT	grounded suite	valid grounded tests	tests with a correct requirement–assertion–evidence trace

11 Experimental Procedure

Figure 5 summarizes the execution order. The suite is frozen before any mutant execution to prevent mutant-specific adaptation.

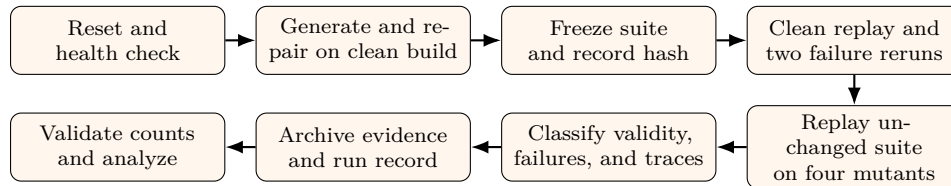


Figure 5: Per-session generation, freeze, replay, classification, and analysis procedure.

Each test receives one primary outcome: pass, application failure, invalid test, flaky, or infrastructure failure. Invalidity categories include syntax/import, selector, impossible precondition, fabricated state or feature, missing setup, absent/vacuous assertion, and unauthorized assumption. A failure is rerun twice on the unchanged build and fixture; alternating outcomes are classified as flaky, consistent with the need to isolate nondeterminism in automated tests [7].

12 Statistical Analysis Plan

For each metric, the report will provide the number of suites, mean, median, standard deviation, and interquartile range by application and condition. Primary comparisons use paired application–replicate differences for RC and

DDR. The analysis will report the paired median difference, a bootstrap 95% confidence interval, and a two-sided paired permutation test. Holm correction applies to the two primary tests. Cliff’s delta is reported as a sensitivity effect size. Secondary outcomes and per-application comparisons are exploratory and will be identified as such. Missing and infrastructure-failed runs remain visible; model or test failures are not rescheduled.

13 Results

No experiment result can be reported from the supplied material. The earlier manuscript explicitly stated that no runs had been conducted, and the current artifact contains zero schema-conforming run records. Table 8 distinguishes fixed design quantities from observations.

Table 8: Evidence status at the 29 September 2026 audit.

Item	Planned	Observed	Evidence status
Applications	3	0	Names and scope available; source/builds absent
Requirements	24	24	Statements extracted and versioned
Atomic behaviors	50	0 executed	Semantic inventory formalized; no execution evidence
Mutants	12	0 implemented	Specifications formalized; patches and reference kills absent
Generation sessions	60	0	No model outputs or generated suites
Clean/mutant executions	≥ 300 suite-build replays		0 No Playwright records, traces, or logs

13.1 Overall results

RC, FC, GSR, ESR, DDR, FPR, ITR, and RT are not observed. Planned counts are not substituted for outcomes.

13.2 TaskBoard

No TaskBoard source, suite, clean execution, or mutant execution was supplied.

13.3 InventoryDesk

No InventoryDesk source, suite, clean execution, or mutant execution was supplied.

13.4 HelpQueue

No HelpQueue source, suite, clean execution, or mutant execution was supplied.

13.5 Requirement coverage

No generated test can be mapped to the 24 requirements; RC is therefore unreported rather than set to zero.

13.6 Behavioral coverage

The 50-behavior denominator is fixed, but no valid tests exist for a numerator; FC is unreported.

13.7 Test validity and execution success

No generated tests or browser outcomes exist. GSR, ESR, and ITR are unreported.

13.8 Mutant detection

No mutant patches or replay evidence exist. The detection matrix cannot be populated and DDR is unreported.

13.9 False positives and invalid tests

No clean-build failure reports exist; FPR and invalidity frequencies are unreported.

13.10 Requirement traceability

The schema and grounded prompt require trace links, but no generated grounded suite exists; RT is unreported.

13.11 Failure analysis

The only observed failures concern artifact availability and environment readiness, not agent behavior: missing application sources, missing experiment implementation, missing empirical records, and missing JavaScript/browser runtime components.

14 Failure Analysis Codebook

Once executions exist, failures will be coded as requirement interpretation, missing behavior, incomplete workflow, state-management error, selector error, navigation error, authentication/role error, assertion/oracle weakness, execution failure, test-generation failure, environment failure, or traceability/reporting failure. Each coded instance must cite a test, log or trace, build, and adjudication note. Categories may be refined inductively, but post-hoc additions will be marked and versioned. The present artifact audit is categorized separately so that missing infrastructure is not confused with poor agent performance.

Table 9: Failure-analysis codebook. Frequencies remain unreported until run evidence exists.

Category	Coding criterion
Requirement interpretation	Test encodes a meaning inconsistent with the approved statement.
Missing behavior	No valid test addresses an approved requirement or atomic behavior.
Incomplete workflow	Intended scenario omits a necessary precondition, action, or verification step.
State management	Test leaks, assumes, or corrupts persistent application state.
Selector/navigation	Locator or route does not identify the intended user-visible target.
Authentication/authorization	Actor, session, credential, or role setup is incorrect.
Assertion/oracle weakness	Assertion is absent, vacuous, implementation-derived, or unrelated to the requirement.
Execution/generation	Syntax, import, timeout, browser, server, or model-generation failure prevents a verdict.
Traceability/reporting	Requirement ID, verdict, or evidence path is missing or inconsistent.

15 Discussion

The research questions remain unanswered. There is no basis for claiming that grounding improves or worsens coverage, validity, traceability, or fault detection. This negative reporting constraint is scientifically important: a complete protocol, plausible requirements, or a polished manuscript is not empirical evidence.

The design nevertheless exposes useful interpretation patterns. If grounding later raises RC and FC without DDR, it would indicate broader specification contact but possibly weak or redundant oracles. If DDR rises together with FPR, grounding may improve sensitivity at a cost to specificity. Higher RT without higher semantic coverage would support auditability but not stronger testing. Grounding may also harm execution by producing longer workflows or over-literal tests when requirements are ambiguous. Conversely, baseline exploration may discover useful behavior not stated in the approved requirements; such tests should be retained and discussed rather than automatically dismissed.

Application-level results are essential. Authorization behavior in InventoryDesk and HelpQueue may benefit more from explicit requirements than visually discoverable TaskBoard controls. Persistence and confirmation behaviors

require multi-step oracles and may reveal differences that test counts obscure. These are hypotheses for interpretation, not findings.

16 Threats to Validity

16.1 Internal validity

Model nondeterminism, prompt sensitivity, provider updates, state leakage, unequal repair decisions, and run-order effects can confound the treatment. One prompt template, interleaved randomization, fresh contexts, deterministic reset, fixed budgets, suite freezing, and full provenance reduce these risks. The requirements necessarily add tokens; padding the baseline with irrelevant text would create a different confound.

16.2 Construct validity

Requirement coverage can reward shallow assertions, so RC requires a relevant action and oracle and is complemented by atomic FC. Traceability measures explicit links, not semantic depth. Mutant detection depends on the representativeness and non-equivalence of 12 seeded faults. False-positive classification depends on the clean build and adjudication. The 50-behavior inventory was formalized before execution but still reflects researcher judgment.

16.3 External validity

Three small researcher-created React applications cannot represent production systems, third-party integrations, varied accessibility quality, other frameworks, or complex backends. Twelve seeded faults do not approximate the full distribution of field defects. Results from one model, prompt family, browser engine, and collection window require independent replication.

16.4 Conclusion validity

Thirty suites per condition provide repeated observations but only three applications. Tests within a suite and suites within an application are correlated. The analysis therefore uses suite-level paired comparisons, application-stratified reporting, intervals, and effect sizes, and restricts multiplicity correction to two predeclared primary outcomes.

16.5 Reproducibility

Hosted models may remain nondeterministic even with a recorded seed. Exact prompts, raw responses, tool calls, suite hashes, browser artifacts, package lockfiles, application commits, model identifiers, and timestamps are required. The current package records the protocol inputs but cannot reproduce absent executions.

17 Reproducibility and Artifact Audit

Table 10 reports what was actually available. The validation script checks the fixed counts and cross-file identifiers; the analysis script refuses to synthesize missing observations and reports completeness against 60 sessions.

Table 10: Artifact audit.

Component	Audit finding	Status
Protocol PDF	13-page protocol; explicitly states that no experiment was conducted	available
LaTeX source		repaired
Requirements	24 statements recoverable from PDF	versioned
Behavior inventory	Count mandated by study scope, but no prior file supplied	formalized pre-experiment
Mutant manifest/patches	No prior manifest or source patches supplied	missing
Applications/reference tests	No source, lockfiles, fixtures, or tests supplied	missing
Agent/runner	No implementation or tool schema supplied	missing
Run records/results/logs	No generated tests, traces, screenshots, or metrics supplied	missing
Runtime	Python and pdfTeX available; Node.js, npm, Playwright, and Chromium absent	blocked

The exact restoration and execution sequence is documented in `docs/REPRODUCIBILITY.md`. Dependency versions must be pinned only after the real application artifact is restored; inventing current package versions would create a misleading manifest.

18 Related Work

This work differs from LLM unit-test generation by evaluating browser-level workflows and requirement-derived oracles rather than functions and code coverage [1]. It differs from broad NLP-to-test research by isolating requirement access as a controlled treatment and by retaining browser evidence through clean and mutant replay [2, 3]. It uses agentic action–observation loops motivated by ReAct and controlled web-agent evaluation, but the objective is verification against a specification rather than open-ended task completion [5, 6]. Mutation testing supplies known violations, while clean builds, invalid-test coding, and flakiness controls prevent every failure from being labeled a defect [4, 7].

19 Conclusion

This paper specifies a controlled evaluation of requirement-grounded LLM agents for functional testing of React applications. The design fixes three applications, 24 requirements, 50 atomic behaviors, 12 mutant specifications, two matched conditions, ten replicates per application and condition, and 60 planned generation sessions. It defines the evidence chain, metrics, execution procedure, statistical analysis, failure codebook, and reproducibility records needed to evaluate coverage, validity, traceability, false positives, and defect detection.

The supplied artifact did not contain the applications or experimental evidence, and the available environment could not execute the intended React/Playwright workflow. The measurements therefore indicate only that zero of 60 planned sessions are presently available; they do not indicate poor agent performance and cannot answer the research questions. The principal next step is to restore and freeze the actual application repository, implement and reference-test the mutants, pin the runtime, execute the randomized sessions, and populate the manuscript exclusively from archived records. Future work beyond that study may examine additional models, larger independent applications, evolving requirements, and other frontend frameworks.

References

- [1] M. Schäfer, S. Nadi, A. Eghbali, and F. Tip, “An empirical evaluation of using large language models for automated unit test generation,” *IEEE Transactions on Software Engineering*, vol. 50, no. 1, pp. 85–105, 2024. doi: [10.1109/TSE.2023.3334955](https://doi.org/10.1109/TSE.2023.3334955).
- [2] V. Garousi, S. Bauer, and M. Felderer, “NLP-assisted software testing: A systematic mapping of the literature,” *Information and Software Technology*, vol. 126, article 106321, 2020. doi: [10.1016/j.infsof.2020.106321](https://doi.org/10.1016/j.infsof.2020.106321).
- [3] A. Mustafa et al., “Automated test case generation from requirements: A systematic literature review,” *Computers, Materials & Continua*, vol. 67, no. 2, pp. 1819–1833, 2021. doi: [10.32604/cmc.2021.014391](https://doi.org/10.32604/cmc.2021.014391).

- [4] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo, “The oracle problem in software testing: A survey,” *IEEE Transactions on Software Engineering*, vol. 41, no. 5, pp. 507–525, 2015. doi: [10.1109/TSE.2014.2372785](https://doi.org/10.1109/TSE.2014.2372785).
- [5] S. Yao et al., “ReAct: Synergizing reasoning and acting in language models,” in *International Conference on Learning Representations*, 2023. https://openreview.net/forum?id=WE_vluYUL-X.
- [6] S. Zhou et al., “WebArena: A realistic web environment for building autonomous agents,” in *International Conference on Learning Representations*, 2024. <https://openreview.net/forum?id=oKn9c6ytLx>.
- [7] Q. Luo, F. Hariri, L. Eloussi, and D. Marinov, “An empirical analysis of flaky tests,” in *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*, pp. 643–653, 2014. doi: [10.1145/2635868.2635920](https://doi.org/10.1145/2635868.2635920).
- [8] Microsoft, “Playwright documentation: Locators,” accessed 29 September 2026. <https://playwright.dev/docs/locators>.
- [9] Microsoft, “Playwright documentation: Trace viewer,” accessed 29 September 2026. <https://playwright.dev/docs/trace-viewer>.